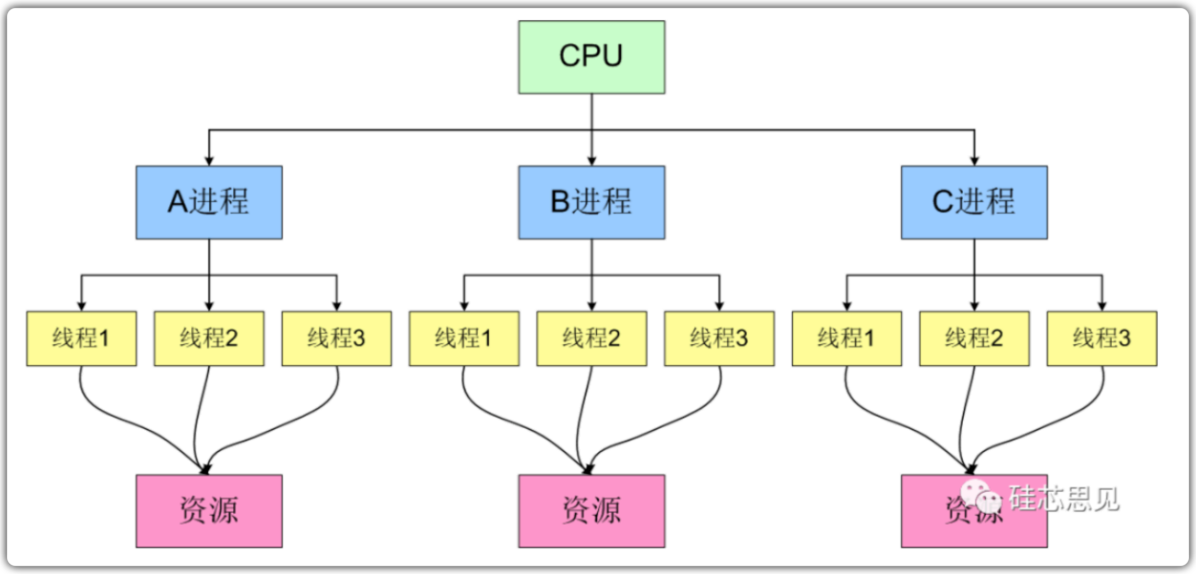
进程（线程）精细化控制中方法的使用

进程/线程

- 在计算机中存在进程和线程的概念，其中进程是并发执行的程序在执行过程中分配和管理资源的基本单位，线程是进程的一个执行单元，是比进程还要小的独立运行的基本单位。
- 在一个程序中至少有一个进程，一个进程至少有一个线程。进程是资源分配最小单位，线程是程序执行的最小单位。
- 计算机在执行程序时，会为程序创建相应的进程，进行资源分配时，是以进程为单位进行相应的分配。每个进程都有相应的线程，在执行程序时，实际上是执行相应的一系列线程。



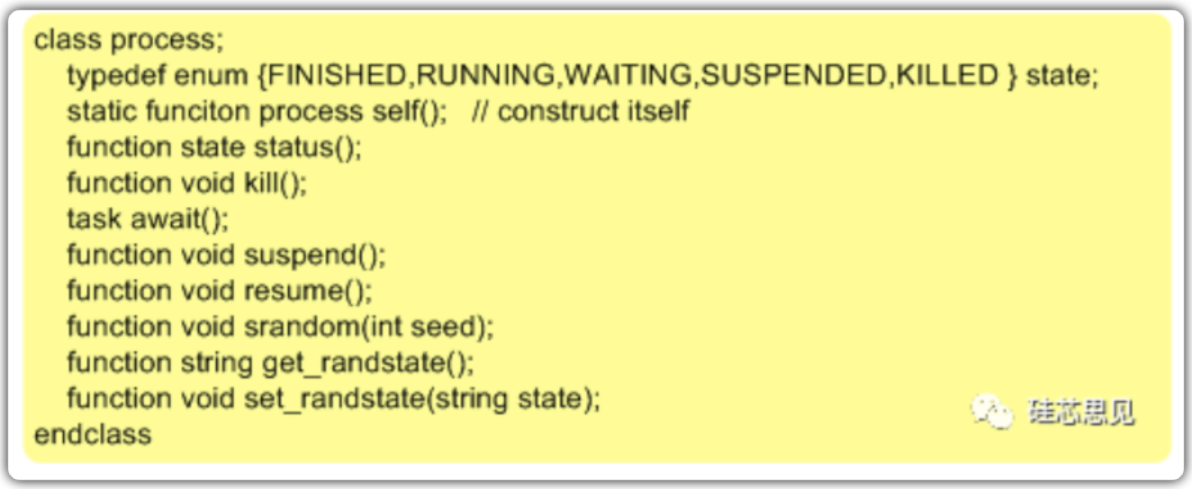
- 在SystemVerilog中，线程的创建主要有以下几种方式：



- 每一个initial结构都会创建一个对应的线程；
- 每一个final结构都会创建一个对应的线程；
- 每一个always、always_comb、always_latch和always_ff结构都会创建对应的线程；
- fork-join（join_any/join_none）结构中每条并行执行的语句块；
- 每一个动态线程，一般并行执行的线程并且不会阻塞其他线程或者task中其他语句执行的线程；
- 每一个连续赋值语句；

SystemVerilog中process

- **在SystemVerilog中可以通过process来实现对于上述线程的控制**，而这个process其实是SystemVerilog中的一个内建类，通过该类声明的句柄可以指向特定的线程，从而可以通过该句柄实现对于指向线程的访问控制。



- 虽然process是一个类，但是比较特殊，不能被拓展派生，也就是说process不会有子类的，并且**该类在创建对象时，是不能使用new函数的，process类型句柄指向对象是通过process::self()完成的**，因此如果用户需要通过process句柄指向对应的线程时需要在对应的线程中调用process::self()实现。

- 在process中经常用来实现对于线程控制的方法主要如下：

    - self()返回指向当前线程的句柄;
    - status()返回当前句柄指向线程的状态（包含的状态有：FINISHED、RUNNING、WAITING、SUSPEND和KILLED等状态）;
    - kill()终止当前句柄指向的线程及其开启的所有子线程;
    - await()等待调用其的句柄指向的线程执行完成，注意不能在调用其句柄指向的线程中使用该方法，只能在别的线程中调用该方法用以阻塞调用该方法的句柄指向的线程;
    - suspend()挂起当前句柄指向线程;
    - resume()恢复被挂起的线程;
    - srandom()设置线程的随机种子;

示例一

- 首先声明了两个句柄p1和p2，在两个initial结构中，p1和p2通过调用“process::self()”实现p1和p2分别指向其所在的initial结构开启的线程。

- 在第一个initial结构中“@0”时刻第一个$display中通过p1.status()方法获取了当前线程的状态(log中显示为RUNNING)，然后等待10个时间单位后，再次调用$display()，在其中调用p1.status()方法获取了当前线程时刻“@10”的状态(log中显示为RUNNING)，再经过100个时间单位，在时刻@110再次在$display()中调用p1.status()方法获取了当前线程的执行状态(log中显示为RUNNING)，该语句执行完后第一个initial结构开启的线程执行完毕。

- 第二个initial结构中，“@0”时刻第一个$display中通过p2.status()方法获取了当前线程的状态(log中显示为RUNNING)，然后等待10个时间单位后，再次调用$display()，在其中调用p2.status()方法获取了当前线程时刻“@10”的状态(log中显示为RUNNING)，然后通过p1调用await()方法，p1.await()后的语句将处于等待p1指向线程执行完毕，其后的语句此时处于阻塞的状态，当p1指向的线程(也就是第一个initial结构)在时刻“@110”执行完毕，p1.await()不再阻塞其后语句的执行，其后的语句继续执行，此时调用p1.status()显示的值为p1当前的状态(log中显示此时p1状态位FINISHED)。在等待100个时间单位，在时刻“@210”调用p2.status()时，p2指向的线程还没有执行完毕，所以其显示的状态仍为RUNNING。

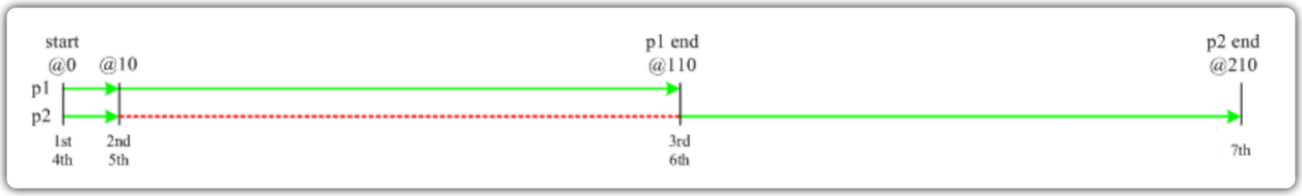
```
`timescale 1 ns / 1 ps
module top_tb;
process p1,p2;
initial begin
    p1 = process::self();  // return a handle to the current thread
    $display("@%0t : 1st : p1 status is %s",$realtime,p1.status());
    #10;
    $display("@%0t : 2nd : p1 status is %s",$realtime,p1.status());
    #100;
    $display("@%0t : 3rd : p1 status is %s",$realtime,p1.status());
end

initial begin
    p2 = process::self();
    $display("@%0t : 4th : p2 status is %s",$realtime,p2.status());
    #10;
    $display("@%0t : 5th : p2 status is %s",$realtime,p2.status());
    p1.await();  // block until p1-thread finished
                 // can not call await() with p2-handle
    $display("@%0t : 6th : p1 status is %s",$realtime,p1.status());
    #100;
    $display("@%0t : 7th : p2 status is %s",$realtime,p2.status());

end
endmodule
```

硅芯思见

```
@0 : 1st : p1 status is RUNNING
@0 : 4th : p2 status is RUNNING
@10000 : 2nd : p1 status is RUNNING
@10000 : 5th : p2 status is RUNNING
@110000 : 3rd : p1 status is RUNNING
@110000 : 6th : p1 status is FINISHED
@210000 : 7th : p2 status is RUNNING
Simulation complete, time is 210000 ps
```



示例二

- 声明了三个process句柄p1、p2和p3.在initial结构中调用“process::self()”实现p1指向当前的initial结构开启的线程，然后调用$display语句通过p1.status()获取当前initial结构开启线程的执行状态为RUNNING(该initial结构正在执行中)。
- fork-join_none结构中所有语句是并行执行的，示例中的fork结构中一共有三个线程，第一个线程通过调用“process::self()”实现p2指向自己，当该方法结束后，该线

程的历史使命也就结束了，这里需要注意p2并没有指向fork。

- 第二个线程为一个begin-end结构，在begin-end中首先通过调用“process::self()”实现了p3指向该线程，然后调用$display中通过p3.status()获取当前线程执行状态为RUNNING(该begin-end结构还在执行中)，在等待100个时间单位，通过p2.status()和p3.status()获取当前时刻p2和p3指向线程的执行状态，因为p2指向的线程在调用“process::self()”完后就执行完了，所以p2.status()返回的状态位FINISHED，p3.status()获取的状态值仍为RUNNING(其所指向begin-end结构还在执行中)，再等待100个时间单位，通过p2.status()和p3.status()获取当前时刻p2和p3指向线程的执行状态，因为此时fork-join_none中并行执行的第三个线程通过p3调用了kill()方法，所以begin-end结构对应的线程被kill掉，因此此时企图在等待100个时间单位再次获取p2和p3执行状态的语句不会执行。

- 第三个线程为等待190个时间单位后调用p3.kill()，实现了对于fork-join_none中第二个线程的kill操作。

- fork-join_none结构后通过p1.status()获取p1指向线程的执行状态为RUNNING(因为p1指向的initial结构正在执行中)；

- 在经过10个时间单位，通过p2.status()和p3.status()获取当前时刻p2和p3指向线程的执行状态分别为FINISHED(p2指向线程已经执行完毕)和WAITING(p3指向线程正在执行其中#100，处于延迟阻塞等待状态)；

- 通过p2.kill()企图kill掉p2指向的线程，实际上此时p2指向的线程已经执行完毕，所以并不会影响p2指向线程的状态，所以此时在p2.kill()后通过p2.status()和p3.status()获取当前时刻p2和p3指向线程的执行状态仍为FINISHED(p2指向线程已经执行完毕)和WAITING(p3指向线程还在执行其中#100，处于延迟阻塞等待状态)；

- 在经过210个时间单位，通过p1.status()、p2.status()和p3.status()获取当前时刻p1、p2和p3指向线程的执行状态为RUNNING(p1指向的initial结构正在执行中)、FINISHED(p2指向线程已经执行完毕)和KILLED(p3指向的线程执行完之前，在fork-join_none结构中等待190个时间单位时已经被kill掉了，所以此时状态位KILLED)；

```
`timescale 1 ns / 1 ps
module top_tb;
// declaration process
process p1,p2,p3;

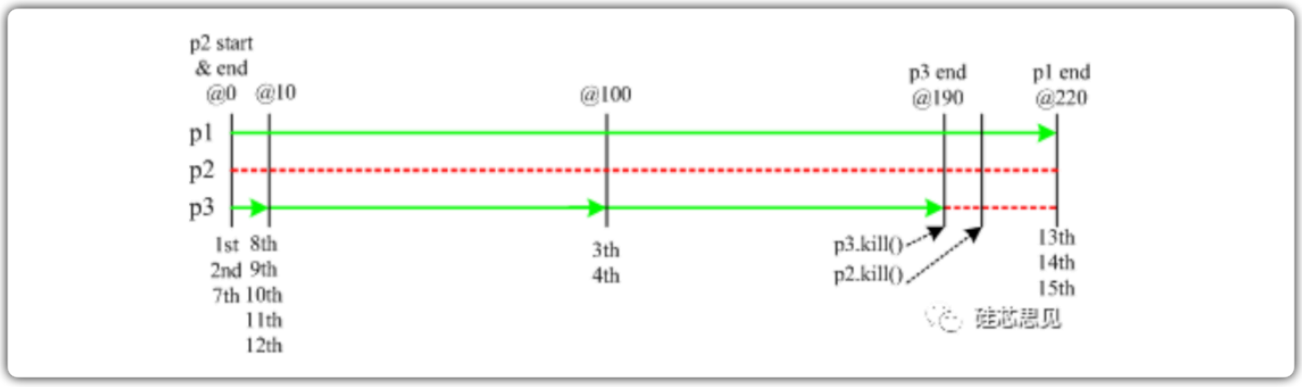
initial $timeformat(-9,0,"ns",10);

initial begin // process
    p1 = process::self();     // return a handle to the current process
    $display("@%0t : 1st : p1 status is %s",$realtime,p1.status());
    fork
        p2 = process::self(); // thread-1 in fork
        begin
            p3 = process::self();
            $display("@%0t : 2nd : p3 status is %s",$realtime,p3.status());
            #100 $display("@%0t : 3rd : p2 status is %s",$realtime,p2.status());
                 $display("@%0t : 4th : p3 status is %s",$realtime,p3.status());
            #100 $display("@%0t : 5th : p2 status is %s",$realtime,p2.status());
                 $display("@%0t : 6th : p3 status is %s",$realtime,p3.status());
        end
        #190 p3.kill();       // thread-3 in fork
    join_none
    $display("@%0t : 7th : p1 status is %s",$realtime,p1.status());
    #10;
    $display("@%0t : 8th : p2 status is %s",$realtime,p2.status());
    $display("@%0t : 9th : p3 status is %s",$realtime,p3.status());
    p2.kill();
    $display("@%0t : 10th : p1 status is %s",$realtime,p1.status());
    $display("@%0t : 11th : p2 status is %s",$realtime,p2.status());
    $display("@%0t : 12th : p3 status is %s",$realtime,p3.status());
    //p1.kill();
    #210;
    $display("@%0t : 13th : p1 status is %s",$realtime,p1.status());
    $display("@%0t : 14th : p2 status is %s",$realtime,p2.status());
    $display("@%0t : 15th : p3 status is %s",$realtime,p3.status());
end
endmodule
```

硅芯思见

```
@0ns : 1st : p1 status is RUNNING
@0ns : 7th : p1 status is RUNNING
@0ns : 2nd : p3 status is RUNNING
@10ns : 8th : p2 status is FINISHED
@10ns : 9th : p3 status is WAITING
@10ns : 10th : p1 status is RUNNING
@10ns : 11th : p2 status is FINISHED
@10ns : 12th : p3 status is WAITING
@100ns : 3rd : p2 status is FINISHED
@100ns : 4th : p3 status is RUNNING
@220ns : 13th : p1 status is RUNNING
@220ns : 14th : p2 status is FINISHED
@220ns : 15th : p3 status is KILLED
Simulation complete, time is 220000 ps.
```

硅芯思见



UVM中对于线程的控制

- 当我们执行run_test时，实际上在run_test这个task中声明了一个process句柄phase_runner_proc，这个句柄在425行通过“process::self()”指向了当前fork-join_none结构中的一个线程，这个线程就是fork-join_none中的begin-end。当后续的m_run_phases()执行完成后,wait等到了m_phase_all_done拉高，表明当前的run_phases执行完毕，**然后通过phase_runner_proc.kill()将fork-join_none中开启的线程及其所有子线程都kill掉，相当于打扫战场，从而通过process实现了对于fork-join_none中线程及其所有子线程的控制操作。**

```
325 // run_test
326 // --------
327
328 task uvm_root::run_test(string test_name="");
329
330   uvm_factory factory= uvm_factory::get();
331   bit testname_plusarg;
332   int test_name_count;
333   string test_names[$];
334   string msg;
335   uvm_component uvm_test_top;
336
337   process phase_runner_proc; // store thread forked below for final cleanup
338

422   // phase runner, isolated from calling process
423   fork begin
424     // spawn the phase runner task
425     phase_runner_proc = process::self();
426     uvm_phase::m_run_phases();
427   end
428   join_none
429   #0; // let the phase runner start
430
431   wait (m_phase_all_done == 1);
432
433   // clean up after ourselves
434   phase_runner_proc.kill();
435
436   report_summarize();
437
438   if (finish_on_completion)
439     $finish;
440
441 endtask
```

硅芯思见